Restaurant Reservation Management System

Duration

48 hours

Experience Level

Junior Full-Stack Developer (6 months – 1 year)

Expected Technology Stack

- Frontend: React
- Backend: Node.js with Express
- Database: MongoDB
- Authentication: JWT or equivalent

Objective

The objective of this assignment is to assess your understanding of **full-stack development fundamentals**, including backend API design, database modeling, frontend integration, role-based access control, deployment, and business logic implementation.

You are required to design and implement a **Restaurant Reservation Management System** that supports both **customer-facing functionality** and **administrative management** of reservations.

Problem Description

Restaurants require a system to manage table reservations efficiently while allowing administrators to oversee and control bookings.

The system should:

- Allow customers to reserve tables for specific dates and time slots
- Prevent double bookings and capacity conflicts
- Provide administrators with visibility and control over reservations

The focus is on **correctness, clarity, and maintainability**, rather than visual design.

Functional Expectations

1. User Roles & Access

The system should support at least two roles:

- **Customer (User)**
- **Administrator (Admin)**

Authentication is required, and access to functionality should be restricted based on role.

---

2. Customer Functionality

Authenticated customers should be able to:

- Create a reservation

- View their own reservations
- Cancel their reservations

Each reservation should include:

- Reservation date
- Time slot
- Number of guests
- Assigned table

---

3. Administrative Functionality

Administrators should be able to:

- View all reservations
- View reservations by date
- Cancel or update any reservation
- Manage restaurant tables (optional but recommended)

The admin interface should clearly distinguish itself from the customer view.

---

4. Restaurant Setup

You may assume:

- A single restaurant
- A fixed number of tables
- Each table has a defined seating capacity

Tables may be:

- Seeded in the database
- Configured programmatically

---

5. Availability & Validation Logic

The system must:

- Prevent overlapping reservations for the same table
- Ensure table capacity meets the number of guests
- Handle invalid booking attempts gracefully

This is a **key evaluation area** of the assignment.

Frontend Expectations

- A simple, functional user interface is sufficient
- Separate or role-aware views for:
    - Customer
    - Administrator
- Clear user flows and meaningful error messages

Backend Expectations

- RESTful API design
- Role-based authorization
- Proper use of HTTP status codes
- Input validation and centralized error handling
- Secure use of environment variables
- Clear and maintainable project structure

Deployment Requirement (Mandatory)

The application **must be deployed** and accessible via a public URL.

- Frontend and backend may be deployed separately or together
- Any standard deployment platform may be used (e.g., Render, Railway, Vercel, Netlify)
- The deployed application should be functional and accessible at the time of review

---

Data Modeling

You are free to design the schema as you see fit.
Typical entities may include:

- User (with role)
- Table
- Reservation

Key design decisions should be briefly documented.

Non-Functional Expectations

- Clean, readable, and well-structured code
- Meaningful naming conventions

- Logical commit history
- No hard-coded secrets or credentials

Submission Guidelines

Please submit:

1. **GitHub repository link** containing the complete source code
2. **Live deployment URL** of the application
3. **README.md** including:
    - Setup instructions
    - Assumptions made
    - Explanation of reservation and availability logic
    - Explanation of role-based access (User vs Admin)
    - Known limitations
    - Areas for improvement with additional time

Evaluation Criteria

Submissions will be evaluated based on:

- Reservation availability and conflict handling
- Role-based access control implementation
- Backend API and data modeling
- Frontend integration and role-specific views
- Deployment quality and stability
- Code quality and documentation clarity

Notes

- A polished UI is not required.
- Advanced features such as payments, notifications, or real-time updates are out of scope.
- Focus on delivering a working, well-structured solution within the given timeframe.